School of Engineering

Course: Diploma in Electronic and Computer Engineering
Diploma in Aerospace Systems & Management

Learning Unit : EG2E55/EG2C68 Microcontroller Programming & Interfacing

Experiment No: Lab 06

Experiment Title: Programming RS232C Serial Communication Port

Objective: To learning programming RS232C Serial Comm Port.

1. The Serial Communication Setup

Figure 1-1 shows how to setup a RS232C serial communication link between a microcontroller board and the PC.

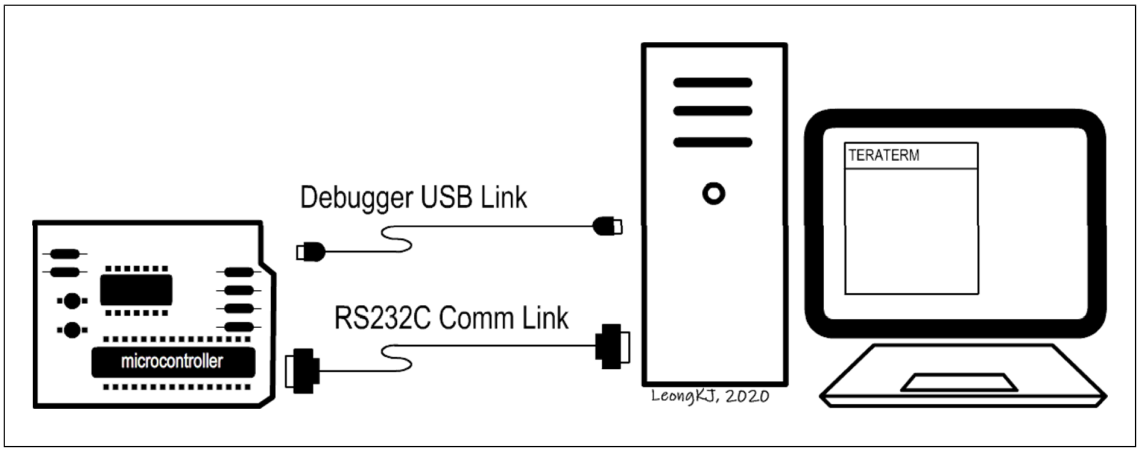


Figure 1-1: The Serial Communication Setup

Setup Teraterm, a serial communication application, on the PC as shown in Figure 1-2 to Figure 1-4. Select the PC serial comm port that is connected to the target board serial comm port.

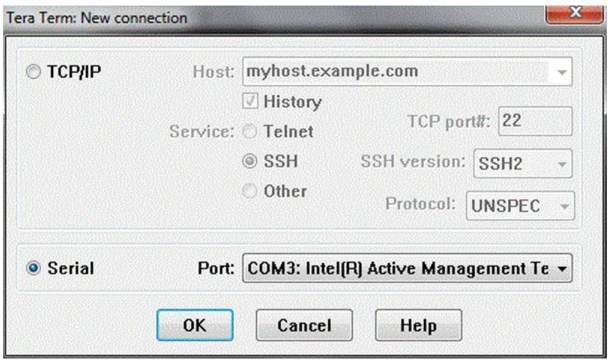


Figure 1-2

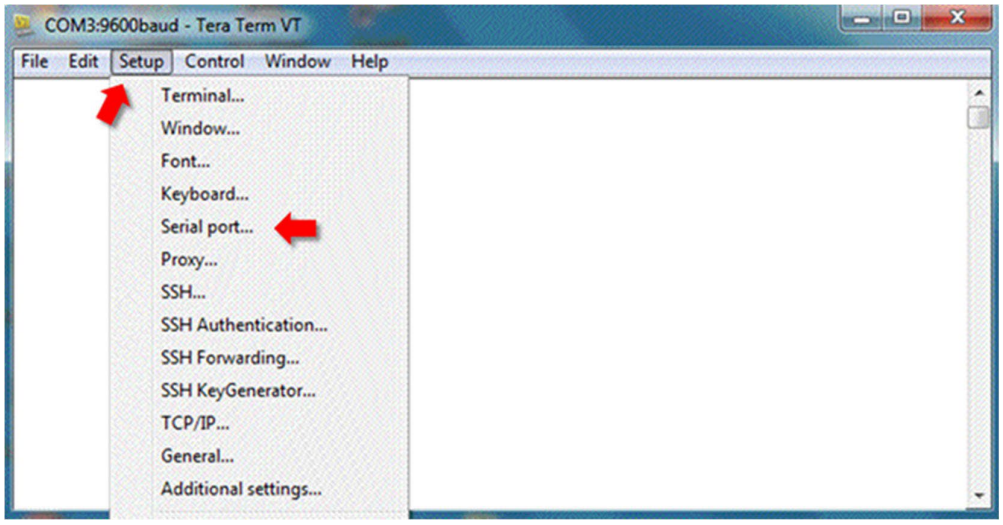


Figure 1-3

Configure the serial port to 9600bps, 8 data bit, no parity, and 1 stop bit (see figure 1-4).

Click “OK” when done.

Your serial terminal application is ready for use.

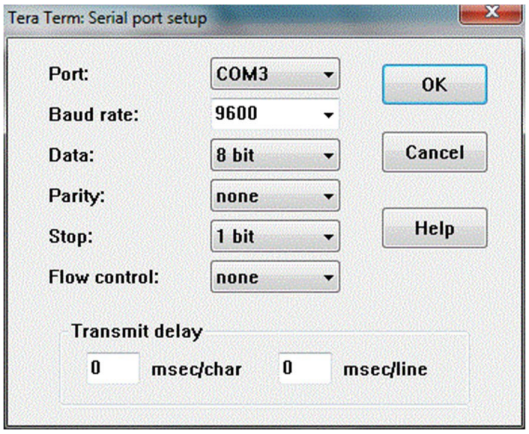


Figure 1-4

2. The RS232C Serial Comm Circuit

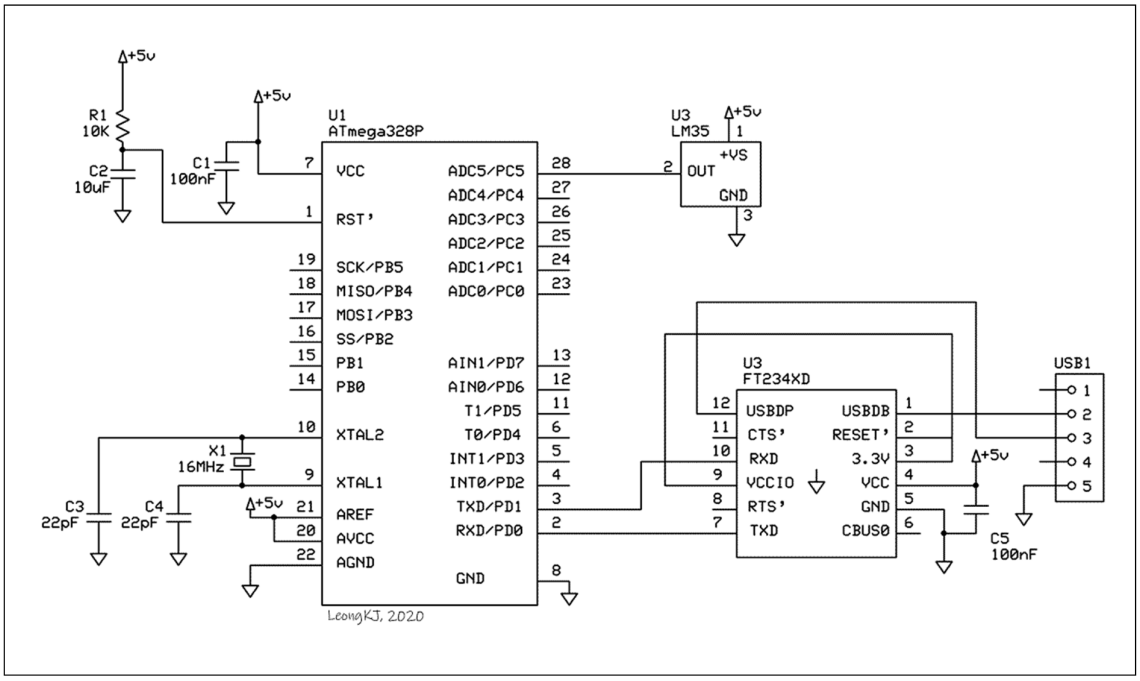


Figure 2-1

3. Serial Transmission Program

The program in Listing 3-1 is supposed to send a string of characters (see Figure 3-1) of "0" (*ASCII 0x30*) to "Z" (*ASCII 0x5A*) repeatedly.

Complete, compile, and execute the program. Before executing the program, make sure that the Teraterm application is already running on the PC.

```
void serialInit(){
  // Select the serial port pins.
  // Setup baud rate.
}

int main(){
  unsigned char w;
  serialInit();
  while(1){
    for(w=0x30;w<0x5B;w++){
      // Check transmit buffer.
      // Send data.
    }
  }
}
```

Listing 3-1

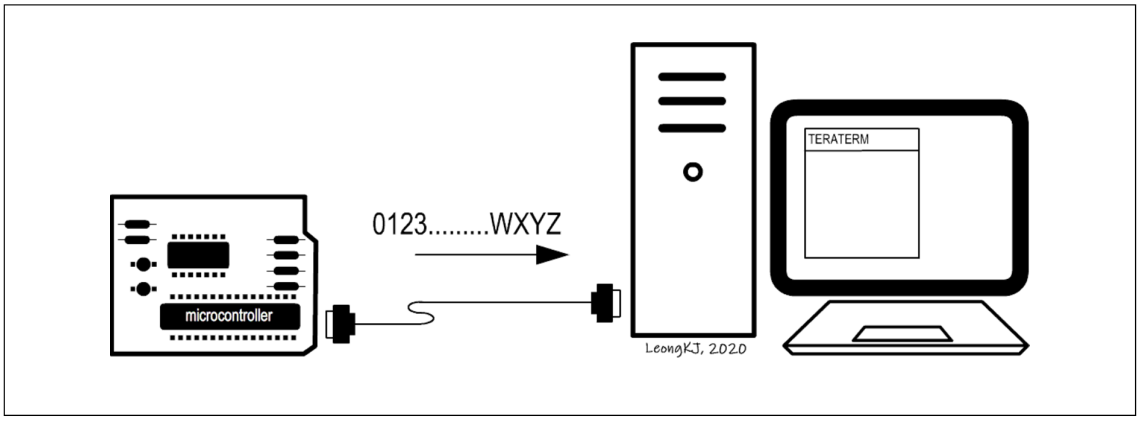


Figure 3-1

4.1 Exercise 1

Your completed program for Listing 3-1 will result in a series of character strings of "0" to "Z" to be sent to the PC and displayed as shown in Figure 4-1.

```
0123456789ABCDEFGHIJKLMNOPQR
STUVWXYZ0123456789ABCDEFGHIJ
KLMNOPQRSTUVWXYZ0123456789AB
CDEFGHIJKLMNOPQRSTUVWXYZ
```

Figure 4-1

Re-write the program in Listing 3-1 by using a function *sendData(x)* to send the data x out of the serial comm port.

```
for(w=0x30;w<0x5B;w++){
  // Check transmit buffer.
  // Send data.
}
```

→

```
for(w=0x30;w<0x5B;w++){
  sendData(w);
}
```

Listing 4-1

4.2 Exercise 2

Modify your program so that every new string of "0" to "Z" starts on a new line (as shown in Figure 4-2).

Hint: Send a *Line-Feed* and *Carriage-Return* characters. Check the ASCII table for their ASCII codes.

```
0123456789ABCDEFGHIJKLMNOPQR
STUVWXYZ
0123456789ABCDEFGHIJKLMNOPQR
STUVWXYZ
0123456789ABCDEFGHIJKLMNOPQR
STUVWXYZ
```

Figure 4-2

4.2 Exercise 3

Change the communication baud rate to 300bps. Observe how slow the data transfer has become.

5. The Serial Communication Setup

Figure 1-1 shows how to setup a RS232C serial communication link between a microcontroller board and the PC.

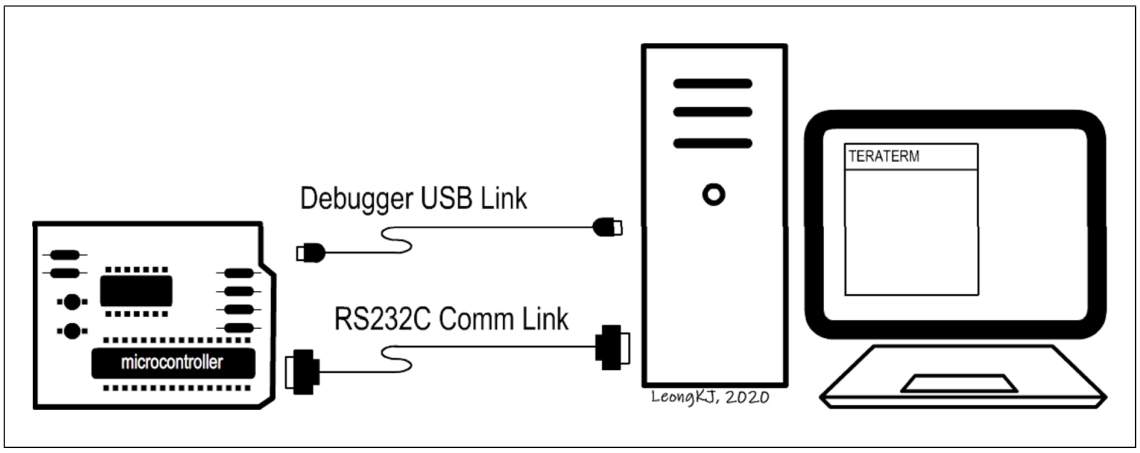


Figure 5-1: The Serial Communication Setup

Setup Teraterm, a serial communication application, on the PC as shown in Figure 1-2 to Figure 1-4. Select the PC serial comm port (*usually COM1*).

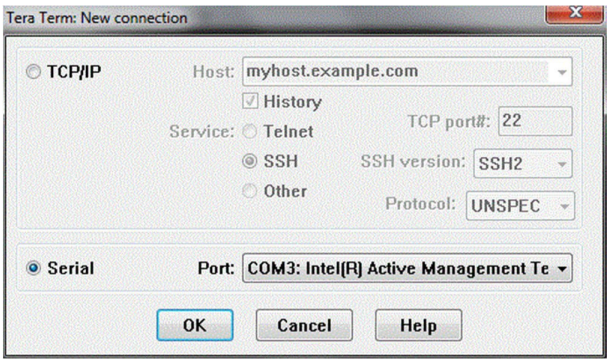


Figure 5-2

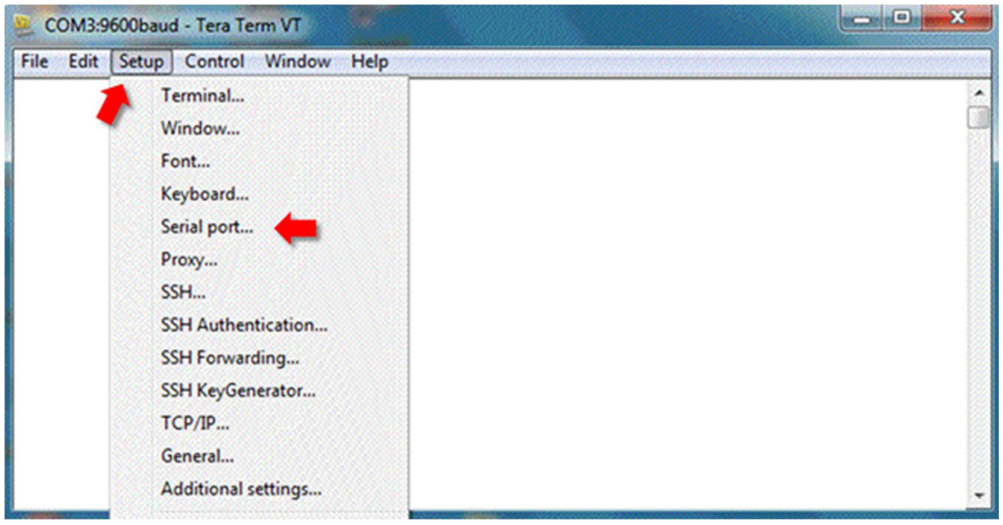


Figure 5-3

Configure the serial port to 9600bps, 8 data bit, no parity, and 1 stop bit (see figure 1-4).

Click “OK” when done.

Your serial terminal application is ready for use.

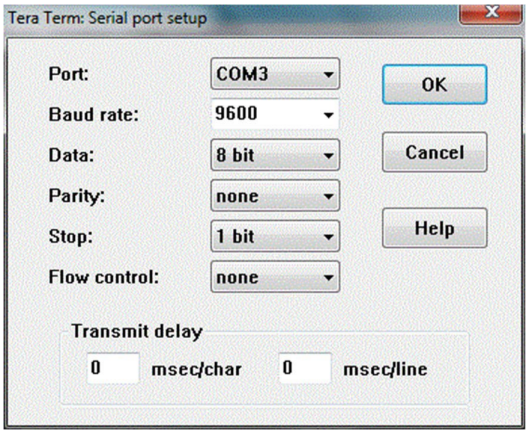


Figure 5-4

6. The RS232C Serial Comm Circuit

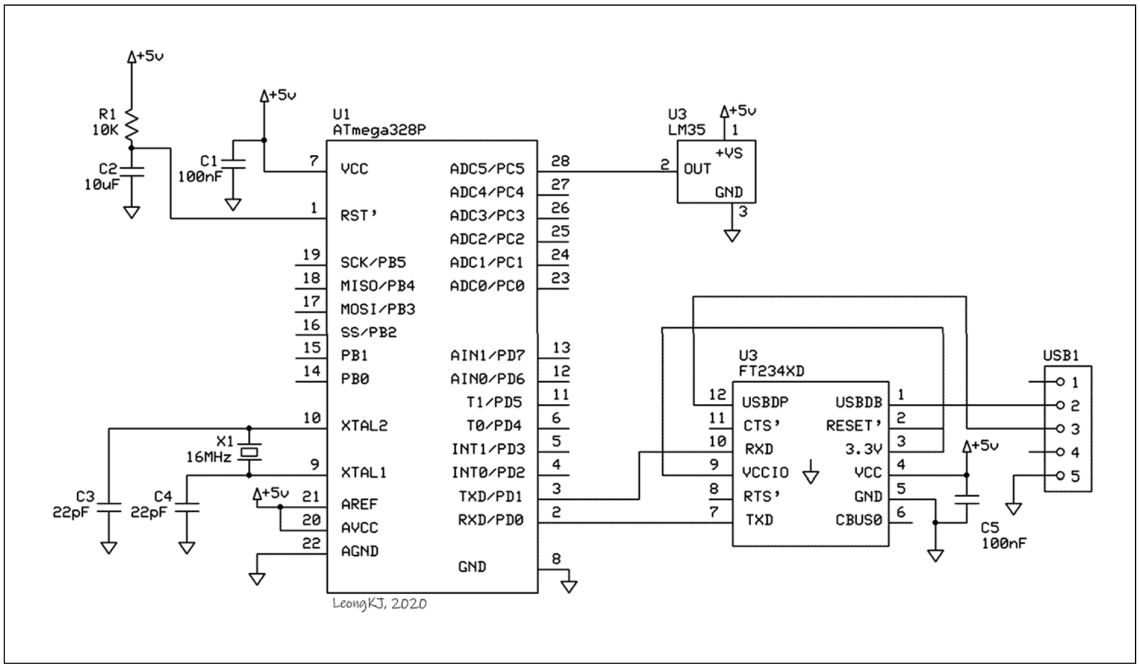


Figure 6-1

7. Sending Serial Information through Serial Comm

Complete the program in Listing 7-1 which sends the temperature acquired by the microcontroller controller ADC channel to the PC.

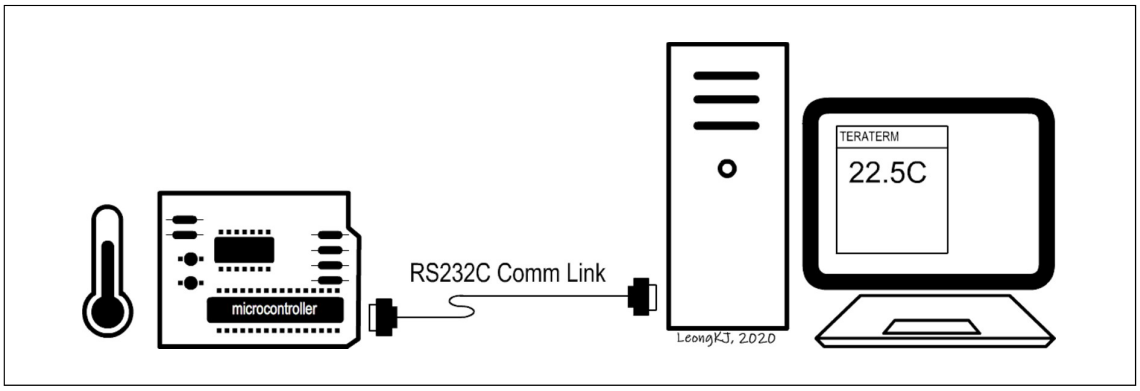


Figure 7-1

```
 #include <avr/io.h>

 void delay(unsigned long k){
while(--k!=0);
 }

 void sendtemp(unsigned int num){
unsigned char n[7];

// Convert the ADC value to
// temperature.

// Break up the number & convert
// them to ASCII code & place
// them in the array.

n[2] = '.';
n[4] = 'C';
n[5] = 0x0A;
n[6] = 0x0D;

// Convert the digit to send
// to ASCII characters.

for(x=0;x<7;x++){
        sendData(n[x]);
}
 }
```

```
void ADC_Init(){
   DIDR0 = 0x20;   // Disable digital.
   DDRC = DDRC & ~(0x20);
   ADCSRA |= 0x87; // Set clock.
   ADMUX = 0xC5; // Vref=1.1V
}

unsigned int ADC_Read(){
   ADCSRA |= 0x40; // Start ADC.
   while ((ADCSRA & 0x10)==0); // Wait.
   ADCSRA |= 0x10; // Reset.
   return ADCW;
}

void SerialInit(){
   - -
}

int main()
{
  unsigned int adcValue;
  SerialInit();
  ADC_Init();
  while(1){
    adcValue = ADC_Read();
    sendtemp(adcValue);
    delay(200000);
  }
}
```

Listing 7-1

7.1 Exercise 4

Modify the program in Listing 7-1 so that it will only send the temperature to the PC when a "T" command is received from the PC (see Figure 7-1).

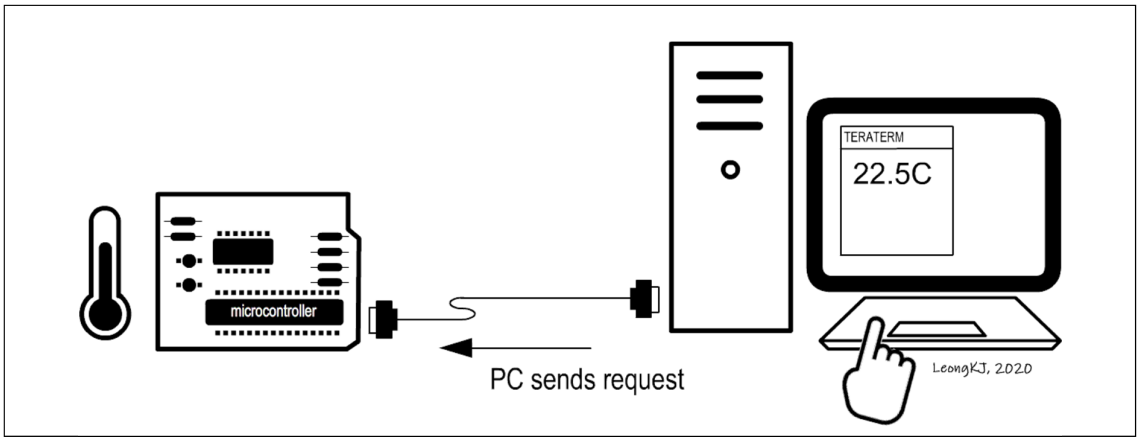


Figure 7-1

7.2 Exercise 5

Provide an option to display the temperature in both Fahrenheit and Celsius. When a 'C' command is received from the PC, the temperature in °C shall be sent to the PC. For °F, the command shall be 'F'. The conversion formula is shown below:

(°C * 1.8) + 32 = °F

Use the hint in Listing 7-2. The sendTemp( ) would have 2 arguments. The second argument indicates whether the temperature is to be in °F or °C.

```
void sendTemp(unsigned int num, unsigned
char r){
  - - -
  // Convert the temperature to °C
  // first. If the request is °F,
  // convert using the given formula.
  - - -
}

int main(){
  - - -
  If new data received {
    Retrieve data to variable rxData.
    adcValue = ADC_Read();
    sendTemp(adcValue,rxData);
  - - -
  - - -
```

Listing 7-2

7.3 Exercise 6

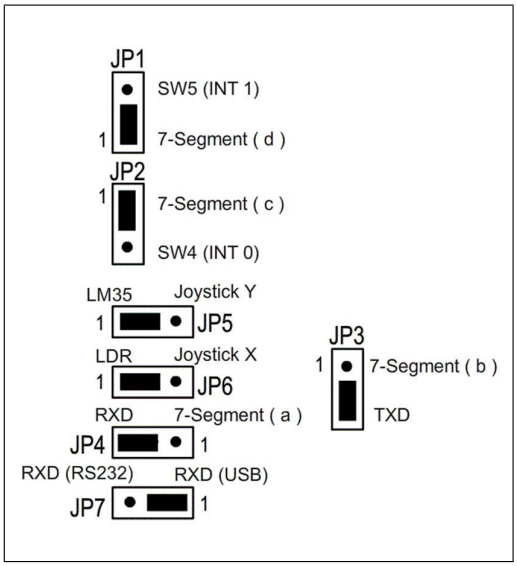
Write a program to echo every character received from the PC back via serial communication at 9600 bps.

In addition, the program shall fulfil the following:

a. If the character received is a digit ('0' to '9'), display its numerical value on the 3-digit display (e.g. receiving '7' displays 007 on the display).
b. If the character received is a letter ('a' to 'z'), convert it to uppercase before echoing back to the PC.
c. All other characters shall be echoed back without modification.

Note: ASCII codes — '0'=0x30, 'A'=0x41, 'a'=0x61. Uppercase conversion: subtract 0x20.

Appendix A - Circuit Board Configuration



END OF PAPER